PYTHON FUNDAMENTALS

LISTS

Practice Workbook

From method fluency to multi-step reasoning
50 focused problems. No filler. No answer key.

15	15	15	5
EASY	MEDIUM	HARD	COMPLEX

A focused progression, not a random question dump

This workbook starts with direct list-method practice, then moves into loops, conditionals, duplicates, ordered data, and multi-step problems that require careful reasoning. The questions are intentionally answer-free so the work stays yours.

50	4	0
problems	difficulty levels	solutions included

How to use this workbook

The goal is not to memorize method names. The goal is to predict how a list changes, choose the right operation, and handle edge cases without losing data.

01-15	Easy Direct practice with append, insert, remove, pop, sort, reverse, and extend.
16-30	Medium Methods combined with conditions, loops, duplicate handling, and preserving order.
31-45	Hard Algorithms involving indexes, contiguous patterns, mutation safety, and complexity-aware reasoning.
46-50	Complex Reasoning Small end-to-end systems: command processors, reconciliation, ranking, and data cleanup.

Rules for your solutions

+ Write each solution as a small function unless the question explicitly asks for a script or processor.

+ Test empty lists, one-item lists, duplicates, missing targets, and boundary values where relevant.

+ Notice whether the task asks you to mutate the original list or return a new one. Those are different requirements.

+ For Hard and Complex questions, follow every restriction. A shortcut that violates a constraint is not a valid solution.

> A good habit: before coding, write down the expected output for one normal case and one edge case. Then trace the list after every mutation.

QUESTIONS 01-15

Easy

Build accuracy with the core list methods. Most tasks should take only a few lines.

Working rule: Use direct list operations wherever possible. Pay attention to whether a method changes the list in place or returns a value.

Q01 Build a packing list

Start with an empty list. Add charger, notebook, water bottle, and headphones in that order using `append()`. Print the final list.

Practice focus: append() and insertion order

Q02 Insert a task into second place

Given `tasks = ['check email', 'attend class', 'revise notes']`, insert submit assignment as the second item. Do not rebuild the list manually.

Practice focus: insert() and zero-based indexes

Q03 Remove one ingredient

Given `spices = ['salt', 'cinnamon', 'pepper', 'cumin']`, remove cinnamon from the list and display the updated list. Assume the item appears exactly once.

Practice focus: remove()

Q04 Pop the latest download

Given `downloads = ['report.pdf', 'notes.txt', 'draft.docx']`, remove the last item with `pop()`. Print both the removed filename and the remaining list.

Practice focus: pop() without an index

Q05 Pop a chosen position

Given `days = ['Mon', 'Tue', 'Wed', 'Thu']`, remove the item at index 2 using `pop()`. Display the removed value and the updated list.

Practice focus: pop(index) and index awareness

Q06 Sort exam scores

Sort `scores = [74, 91, 68, 85, 77]` into ascending order using the list's `sort()` method. Print the same list after sorting.

Practice focus: sort() in place

Q07 Rank delivery times

Given `minutes = [42, 35, 51, 28, 39]`, arrange the times from largest to smallest with `sort()`. Do not create a second list.

Practice focus: sort(reverse=True)

Q08 Reverse recently opened files

Given `files = ['home.py', 'lists.py', 'loops.py', 'main.py']`, reverse the list in place and print it.

Practice focus: reverse()

Q09 Combine today's and tomorrow's tasks

Given `today = ['lecture', 'practice']` and `tomorrow = ['revision', 'quiz']`, add every item from `tomorrow` to the end of `today` in one list-method call. Print `today`.

Practice focus: extend() and list growth

Q10 Append a new reading

A log contains `readings = [28, 30, 29]`. Append the new reading 31, then print the updated list and its number of entries.

Practice focus: append() and len()

Q11 Insert a correction

Given `marks = [12, 18, 25, 31]`, insert 99 as the third item. Preserve the order of all existing marks.

Practice focus: insert() at index 2

Q12 Remove only the first matching value

Given `drinks = ['tea', 'coffee', 'tea', 'water']`, remove one occurrence of tea with `remove()`. Print the list and confirm that the other tea remains.

Practice focus: remove() removes the first match

Q13 Serve the next ticket

A queue is stored as `tickets = ['T-104', 'T-105', 'T-106']`. Remove the first ticket using `pop(0)`. Print the ticket served and the remaining queue.

Practice focus: pop(0)

Q14 Alphabetize filenames

Given `names = ['report.py', 'app.py', 'utils.py', 'main.py']`, sort the lowercase filenames alphabetically and print the result.

Practice focus: sort() on strings

Q15 Apply two operations in order

Start with `status = ['queued', 'running', 'waiting']`. Append complete, then reverse the same list. Print the final order. The order of operations matters.

Practice focus: append() followed by reverse()

QUESTIONS 16-30

Medium

Combine methods with loops, conditions, duplicate handling, and order-preserving logic.

Working rule: Write a small function for each task. Unless stated otherwise, return a result instead of only printing it.

Q16 Process a task queue

Given a queue of pending jobs, append two new jobs to the end, then serve the first two jobs in arrival order using list operations. Return both the jobs served, in order, and the queue that remains. Assume the queue has at least two jobs before serving.

Practice focus: append(), pop(0), and sequence tracking

Q17 Move a selected song to the end

Given a playlist and a selected song that appears exactly once, move that song to the final position. Every other song must keep its relative order. Do not sort the playlist.

Practice focus: remove() + append()

Q18 Rotate right once

Move the last value of a list to the front. For example, `[10, 20, 30, 40]` becomes `[40, 10, 20, 30]`. Use `pop()` and `insert()` or `append()`; do not use slicing. An empty list must remain empty.

Practice focus: pop(), insert(), edge cases

Q19 Delete every occurrence of a blocked value

Given a list that may contain repeated values and a target value, remove every occurrence of the target. Use a loop and list methods; do not use a list comprehension. Make sure adjacent matches are not skipped.

Practice focus: while, in, and remove()

Q20 Keep only positive measurements

Given a list of integer measurements, build and return a new list containing only values greater than zero, in their original order. Do not modify the source list.

Practice focus: for loop, if, append()

Q21 Find the second-highest distinct score

Given a list of scores, return the second-highest distinct value. Repeated top scores count only once. If the list has fewer than two distinct values, return `None`. Do not use a set.

Practice focus: sorting and duplicate checks

Q22 Insert into a sorted list without re-sorting

Given a list already sorted in ascending order and a new number, insert the number at the correct position without calling `sort()`. Place the new number after existing equal values so their order is preserved.

Practice focus: loop + insert()

Q23 Remove invalid exam marks safely

Marks should be between 0 and 100 inclusive. Remove every value outside that range from the original list. Your solution must work when two or more invalid marks are next to each other.

Practice focus: conditions and safe mutation

Q24 Separate adults from minors

Given a list of ages, create two lists: `adults` contains ages of 18 or more, and `minors` contains ages below 18. Preserve the original order within both result lists.

Practice focus: if/else and append()

Q25 Find the first repeated value

Scan a list from left to right and return the first value whose current occurrence has appeared earlier. For `[6, 9, 2, 9, 6]`, the answer is 9, because its second occurrence appears first. Return `None` if no value repeats. Do not use a set.

Practice focus: nested loops or prefix tracking

Q26 Remove duplicates but preserve first appearances

Return a new list in which each value appears once, keeping the first occurrence and original order. For example, `[4, 2, 4, 7, 2]` becomes `[4, 2, 7]`. Do not use a set or dictionary.

Practice focus: membership checks and append()

Q27 Interleave two lists

Combine two lists by taking one item from the first list, then one from the second, repeatedly. If one list runs out, append all remaining items from the longer list. Example: `['A','B','C','D']` and `[1,2]` produce `['A',1,'B',2,'C','D']`.

Practice focus: indexes, loops, and unequal lengths

Q28 Keep only the latest five readings

Start with a list of recent readings and process a second list of new readings in order. After each new reading is appended, remove the oldest item while the stored list has more than five items. Return the final list. It must never contain more than five readings.

Practice focus: append(), pop(0), while

Q29 Return the top three distinct scores

Given a list of scores, return up to three distinct values from highest to lowest. Duplicate scores should appear once in the result. If only one or two distinct values exist, return only those values.

Practice focus: sort(reverse=True) and filtering

Q30 Mark values that exceed a threshold

Given a list of numbers and a threshold, create a new list that places the string `'CHECK'` immediately before every number greater than the threshold. Keep every original number and its order. Example: `[4, 11, 7, 15]` with threshold `10` becomes `[4, 'CHECK', 11, 7, 'CHECK', 15]`.

Practice focus: conditions and building a result list

QUESTIONS 31-45

Hard

Focus on algorithms, mutation safety, contiguous patterns, and precise edge-case behavior.

Working rule: Respect each restriction. A correct result reached by a forbidden shortcut does not satisfy the problem.

Q31 Merge two sorted lists manually

Given two lists already sorted in ascending order, merge them into one sorted list without calling `sort()` and without sorting either input. Preserve duplicates. Use indexes and comparisons to choose the next value, then append any remaining values.

Practice focus: two-pointer merge

Q32 Rotate right by k positions

Rotate a list right by k positions without slicing. For example, `[1,2,3,4,5]` rotated by 2 becomes `[4,5,1,2,3]`. Handle an empty list and values of k larger than the list length.

Practice focus: modulo, pop(), insert()/append()

Q33 Find the earliest longest +1 run

Return the longest contiguous run in which every value is exactly one greater than the value before it. If multiple runs have the same maximum length, return the earliest one. Example: `[4,5,6,2,3,4,9]` returns `[4,5,6]`.

Practice focus: single-pass state tracking

Q34 Measure the longest positive streak

Given a list of numbers, return `(length, start_index, end_index)` for the longest contiguous run of values greater than zero. Use inclusive indexes. If there is no positive value, return `(0, None, None)`. If there is a tie, choose the earliest streak.

Practice focus: loops, conditions, index bookkeeping

Q35 Find local dips

Return the indexes of all interior values that are strictly smaller than both immediate neighbors. The first and last items can never qualify. Example: `[8, 3, 6, 2, 5]` returns indexes `[1, 3]`. Do not change the source list.

Practice focus: neighbor comparisons

Q36 Collapse consecutive duplicates in place

Modify the input list so only one copy remains from each consecutive run. Do not remove later non-consecutive repeats. Example: `[2,2,1,2,2]` becomes `[2,1,2]`. Perform the operation in place; do not create a replacement list.

Practice focus: index-controlled pop() or remove()

Q37 Find a contiguous segment with a target sum

Given a list that may include positive, zero, and negative integers, return the start and end indexes of a contiguous segment whose sum equals a target. Choose the smallest start index; for that start, choose the smallest end index. Return None if no segment matches. Do not assume values are all positive.

Practice focus: nested loops and running sums

Q38 Stable-partition negative, zero, and positive values

Create a new list with all negative values first, then all zeros, then all positive values. Preserve the original relative order within each group. Do not sort the values and do not modify the input list.

Practice focus: multi-bucket list construction

Q39 Run-length encode a list

Convert consecutive runs into pairs of `[value, count]`. Example: `[3,3,3,1,1,4]` becomes `[[3,3],[1,2],[4,1]]`. An empty input should return an empty list. Do not use `count()` for each value.

Practice focus: nested lists and run boundaries

Q40 Keep values that occur exactly once

Return a new list containing only values that occur exactly once in the source. Preserve their original order. Example: `[4,7,4,2,9,7]` becomes `[2,9]`. Do not use sets or dictionaries.

Practice focus: frequency reasoning without extra data structures

Q41 Maintain a top-five live leaderboard

You receive an initial list of scores and then a stream of new scores. Keep the stored list in descending order by inserting each new score at its correct position; do not sort the whole list after every update. After each insertion, remove the final item if the list has more than five scores. Duplicate scores are allowed.

Practice focus: insert(), pop(), ordered insertion

Q42 Compare two event logs

Return the events that appear in the first log but not the second. Keep their first-log order and include each event only once. Example: `['open','save','open','close']` minus `['save']` becomes `['open','close']`. Do not use sets or dictionaries.

Practice focus: membership tests and duplicate suppression

Q43 Test whether a list is a palindrome

Return `True` if the list reads the same forwards and backwards, otherwise return `False`. An empty list counts as a palindrome. Do not reverse the list, create a reversed copy, or use slicing.

Practice focus: two-pointer comparison

Q44 Find the shortest window reaching a target

Given a list of non-negative integers and a positive target, return the minimum length of any contiguous window whose sum is at least the target. Return `0` if no window qualifies. Example: `[2,1,5,2,3,2]` with target `7` returns 2.

Practice focus: sliding window

Q45 Find a balance index

Return the first index where the sum of all values to its left equals the sum of all values to its right. Do not include the value at the index in either sum. Return `None` if no such index exists. Example: `[1,7,3,6,5,6]` returns index 3.

Practice focus: prefix totals and one-pass reasoning

QUESTIONS 46-50

Complex Reasoning

Multi-step problems that combine list methods, loops, conditions, state changes, and edge cases.

Working rule: Treat each problem as a small program. Define clear return values, process commands in order, and test empty lists, repeated values, and invalid input.

Q46 Shopping-cart command processor

Write `process_cart(initial_cart, commands)`. Work on a copy so the caller's original cart is unchanged. Process each command in order: `ADD:item` appends an item; `REMOVE:item` removes its first exact match or records 'not found'; `REMOVE_LAST` pops the final item or records 'empty'; `SORT` sorts the cart alphabetically. Return the final cart and one log entry per command, including whether it succeeded. Test a command sequence that includes a missing product and an empty-cart removal.

Practice focus: copying, append(), remove(), pop(), sort(), logs

Q47 Support-ticket waiting line

Write `process_tickets(waiting, commands)`. Commands are `ARRIVE:id`, `CANCEL:id`, and `SERVE`. New tickets join the end; do not allow the same ID to be waiting twice; cancellation removes a matching ticket if present; serving removes the oldest ticket with `pop(0)`. If a cancel target is missing or the queue is empty, record the event and continue. Return the remaining queue, tickets served in order, and an event log.

Practice focus: append(), remove(), pop(0), if/else, event processing

Q48 Reconcile stock movements

Two parallel lists describe stock changes: `items` contains product names and changes contains matching integer quantity changes at the same indexes. Combine all changes for each product, keeping products in first-seen order. Return two lists of `[product, net_quantity]`: `in_stock` for totals greater than zero, and `needs_review` for totals at or below zero. Assume both input lists have equal length. Use lists, loops, and conditions only - no dictionaries or sets.

Practice focus: parallel lists, insert/update logic, filtering, order preservation

Q49 Clean and rank a leaderboard

Each submission is a string such as `'Asha:84'`. Ignore malformed submissions and scores outside 0-100. For each player, keep only their best valid score. Return rows in descending score order; ties must be alphabetical by player name, ignoring case. Do not use dictionaries or sets. Your solution must handle a player whose later submission is lower, a tie, and an invalid score.

Practice focus: string parsing, nested lists, conditional updates, sorting

Q50 Sensor-data cleanup pipeline

Given a list of integer readings, first discard readings outside the valid range 0-100. Then collapse consecutive duplicate readings, keeping one from each run. On the cleaned list, find the longest contiguous strictly increasing run; if several runs tie, choose the earliest. Return the cleaned readings, the number of invalid readings discarded, and the run as `(start_index, end_index, length)` using indexes in the cleaned list. If the cleaned list is empty, return `None` for the run. Do not sort the readings because their order represents time.

Practice focus: filtering, in-place/list-building choices, duplicate runs, streak logic

What you will learn from this set

Finishing the workbook carefully should give you more than familiarity with list syntax. You should be able to reason about list state and build small, reliable data-processing routines.

1. Core list-method fluency	Know when to use `append()`, `extend()`, `insert()`, `remove()`, `pop()`, `sort()`, and `reverse()`. Understand which methods mutate a list and what value, if any, they return.
2. Indexing and mutation safety	Track zero-based indexes, remove an item from a particular position, and avoid skipping items when deleting repeated or adjacent matches. Distinguish in-place updates from making a new list.
3. Loops and conditional logic	Scan values in order, build filtered results, use `if/else` to classify data, and keep counts, indexes, and state variables accurate.
4. Duplicates and order guarantees	Handle first matches, repeated values, distinct rankings, consecutive duplicates, and unique-only results. Preserve original order when the task requires it instead of sorting everything.
5. Practical list patterns	Practise queue processing, ordered insertion, merging sorted lists, stable partitioning, run-length encoding, two-pointer comparison, sliding windows, and prefix-total reasoning.
6. Multi-step problem solving	Translate a real-ish workflow into explicit rules, process commands in sequence, record failures without crashing, reconcile repeated records, and test boundary cases.

How to get the most out of it

Solve Easy without looking up solutions. For Medium and above, write test cases before you code. After a solution works, explain its logic in plain language and check whether it changes the original list. For the Complex problems, split the task into smaller steps and test each step independently. If you can solve these consistently, you are practising problem-solving patterns that transfer beyond lists.

Progress beats speed. A correct, tested solution is more valuable than a fast guess.